# Contents

INTRODUCTION ..... 2
OBJECTIVES ..... 2
HARDWARE OVERVIEW: ..... 3
Role of Buffer (74LS245): ..... 4
Mode Selection Logic: ..... 5
Software Architecture: ..... 6
Mode 1: Vertical Motion Computation: ..... 7
Mode 2: Cylinder Volume Computation: ..... 9
Mode 3: LED Bar Graph Intensity Controller: ..... 10
Mode 4: Prime / Composite Number Detection: ..... 12
Timing and Delay Design: ..... 15
Overflow Detection Mechanism: ..... 17
HARDWARE IMAGES: ..... 19
CONCLUSION: ..... 21

# INTRODUCTION

This project is based on the 8051 microcontroller and is designed to demonstrate practical understanding of assembly language programming, hardware interfacing, and real-time control using a single unified system. The program operates in multiple modes selected through control pins on Port 3, allowing the same hardware to perform mathematical computations, LED visualization, and logical decision-making without requiring a reset. Inputs are taken through Port 1, while results and visual feedback are displayed using LED banks connected to Ports 0 and 2 via 74LS245 buffer circuits to safely handle current requirements. The project combines arithmetic operations, timing control using internal timers, lookup tables, and conditional branching to meet all functional requirements defined in the course project guidelines. Overall, this work bridges theoretical microprocessor concepts with hands-on embedded system implementation, highlighting efficient use of limited 8-bit resources and structured assembly programming.

# OBJECTIVES

- To design and implement a **multi-mode embedded system** using the 8051 microcontroller with mode selection through Port 3.
- To perform **mathematical computations** using assembly language, including multiplication, division, square calculations, and multi-byte arithmetic.
- To interface **LED banks with the microcontroller** using buffer ICs (74LS245) for safe current driving.
- To display computational results correctly on **Port 0 and Port 2** using low-byte and high-byte representation.
- To implement **overflow detection** and indicate it using dedicated control pins.
- To generate **accurate time delays** using Timer-0 and crystal-controlled clock frequency.

- To design a **LED bar graph intensity controller** using software-based PWM and lookup tables.
- To apply **number theory concepts** by detecting prime and composite numbers using assembly-level algorithms.
- To ensure **immediate response to mode changes** without requiring system reset.
- To reinforce understanding of **structured programming techniques** in 8051 assembly language.

# HARDWARE OVERVIEW:

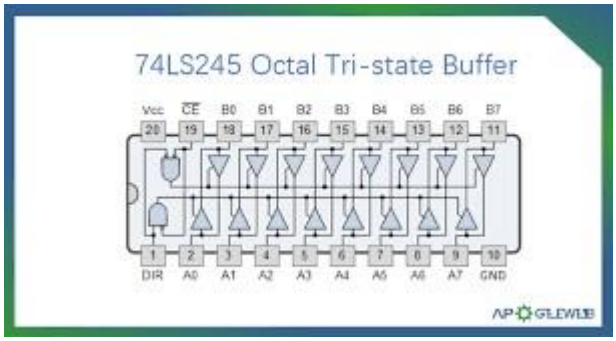
- **Microcontroller:**
  The system is built around the **8051 microcontroller**, operating with an external **11.0592 MHz crystal oscillator**, which ensures accurate timing for delay generation and arithmetic operations.
- **Generic 8051 Development Board:**
  A custom generic board (designed in Project #1) is used, providing connections for Ports 0, 1, 2, and 3, reset circuitry, crystal connections, and external interfacing headers.
- **Input Interface (Port 1):**
  Port 1 is connected to a **DIP switch board** and is used to provide 8-bit input data for mathematical computations and LED control operations.  **Mode Selection Interface (Port 3):**
  Pins **P3.1 and P3.0** are used to select different operating modes, while other Port 3 pins are used for **overflow indication and speed control**.
- **Output Interface (Ports 0 and 2):**
  Ports 0 and 2 drive LED banks to display computational results and visual patterns. Each port controls **8 LEDs**, allowing 16-bit data display.
- **Buffer Circuit (74LS245):**
  Each port is interfaced through a **74LS245 tri-state buffer** to handle higher current requirements and protect the microcontroller pins.
- **Resistor Networks:**

**470 Ω resistors** are used in series with LEDs to limit current, while **10 kΩ pull-up resistor banks** are used for Port 0 due to its open-drain configuration.

- **Power Supply and Reset Circuitry:**
  A regulated **+5 V power supply** powers all components, and a standard reset circuit ensures reliable startup of the microcontroller.

## Role of Buffer (74LS245):






The 8051 microcontroller has limited current sourcing and sinking capability, which makes it unsuitable for directly driving multiple LEDs or other high current loads. To overcome this limitation and to ensure reliable operation, the **74LS245 tri-state buffer** is used between the microcontroller ports and the external LED banks. The buffer acts as an interface that electrically isolates the microcontroller from the load, thereby protecting it from excessive current draw and potential damage.

Key roles of the 74LS245 buffer include:

- Providing **higher current drive capability** required by LED arrays, which the 8051 alone cannot safely supply.
- Protecting the microcontroller pins from overloading by handling most of the current drawn by external devices.
- Allowing **bidirectional data flow** when required, although in this project it is primarily used for output buffering.
- Ensuring **proper logic level matching** between the microcontroller and external components.

- Improving overall **signal integrity and stability**, especially when multiple LEDs switch simultaneously.

In addition, the tri-state nature of the 74LS245 allows clean isolation during reset or unused states, reducing unintended LED flicker. Overall, the use of buffering is essential for building a robust and hardware-safe embedded system, making the design suitable for real-world implementation rather than simulation only.

## Mode Selection Logic:

The project supports four distinct operational modes, selected dynamically using two input pins—**P3.0** and **P3.1**—configured as external mode-select lines. These pins are interfaced with DIP switches on the user-made selection board, allowing the user to change modes in real time without resetting the microcontroller.

The selection logic follows a straightforward **2-bit binary encoding scheme**, where the combination of P3.1 (MSB) and P3.0 (LSB) determines the active mode:

| P3.1 | P3.0 | MODE | FUNCTION |
|---|---|---|---|
| 0 | 0 | 1 | Vertical motion: $h = 5t - 5t^2$ |
| 0 | 1 | 2 | Cylinder volume: $V = 3r^2 \times 10$ |
| 1 | 0 | 3 | LED bar graph intensity mapping |
| 1 | 1 | 4 | Prime/composite detection with LED chasers |

### Implementation in Assembly Code:

The main polling loop (**Check_Conditions**) continuously reads P3.1 and P3.0 using **JNB**/**JB** conditional branching instructions:

```
 9  Check_Conditions:
10
11      JNB P3.1, P3_1_IS_0
12
13      ; P3.1 is 1:
14      JB P3.0, P3_0_IS_1_IN_A
15
16      ; Pattern is (1-0)
17      CALL MODE3
18      JMP Check_Conditions
19
```

```
20  P3_0_IS_1_IN_A:
21
22      CALL MODE4
23      JMP Check_Conditions
24
25  P3_1_IS_0:
26      ; P3.1 is 0:
27      JB P3.0, P3_0_IS_1_IN_B
28
29      ; Pattern is (0-0)
30      CALL MODE1
31      JMP Check_Conditions
```

- **Non-blocking & Immediate Response:** Since the loop contains no **RET** or long blocking calls outside the mode subroutines, any change in P3.0/P3.1 (via DIP switch toggle) is detected **within one loop iteration** (<100 µs at 11.0592 MHz), satisfying the requirement: *"Change over to any mode should take effect immediately."*
- **Debouncing Consideration:** While mechanical DIP switches are immune to bounce (unlike pushbuttons), the continuous polling inherently provides debouncing by ignoring transient glitches due to synchronous reading.

## Software Architecture:

The software is written entirely in **8051 assembly language**, following a **monolithic, polling-based**, and **non-interrupt-driven design**. The architecture prioritizes simplicity, real-time responsiveness, and direct hardware control—consistent with the educational scope and hardware constraints of the 8051 platform.

**Key Structural Features**

- **Single Main Loop:** The program begins at **ORG 0000H** and enters an infinite polling loop (**Check_Conditions**) that continuously reads modeselect inputs (P3.0, P3.1) and dispatches execution to the corresponding mode subroutine.
- **Mode-Driven Modular Subroutines:** Four independent mode routines (**MODE1–MODE4**) encapsulate distinct functionalities. Each handles:
- Input acquisition (from **Port 1**),
- Computation/logic,
- Output formatting (to **Port 0** and **Port 2**), □ Overflow flagging (P3.2/P3.3 where applicable), □ Timed or continuous display/update.
- **Shared Delay Infrastructure:** Reusable delay subroutines (**DELAY_2SEC, DELAY_1_5SEC, DELAY_300MS, DELAY_600MS**)

use **Timer 0 in mode 1 (16-bit)** for accurate timing, configured inline (no global **TMOD** setup), ensuring portability across modes.

- **No Interrupts or Stack-Heavy Constructs:** All control flow uses **CALL/RET** and conditional jumps (**JB, JNB, JZ, JC**). Local data uses registers (R0–R7) and direct/internal RAM (e.g., **30H**), minimizing stack usage.

**Data Flow**

1. **Input:** 8-bit data from user (DIP switches) → **Port 1** (read-only, pulledup).
2. **Mode Select:** Hardwired DIP switches → **P3.0, P3.1** → decoded in real time.
3. **Processing:** Pure 8051 integer arithmetic (add, mul, div, rotate).
4. **Output:**
   - Numerical results → split into **low byte (P0)** / **high byte (P2),**
   - LED patterns → direct port writes (with 74LS245 buffering),
   - Overflow/status → P3.2 (Mode 1), P3.3 (Mode 2), P3.5 (Mode 4 speed control).
5. **Timing:** Each mode controls its own display duration or refresh rate via delay calls.

# Mode 1: Vertical Motion Computation:

Mode 1 is activated when P3.1 = 0 and P3.0 = 0. It computes the vertical height of a projectile launched upward with:

- Initial velocity u = 5 m/s
- Acceleration due to gravity g = $-10\left(\frac{m}{s^2}\right)$

The kinematic equation is:

$$h = u \cdot t - \frac{1}{2}gt^2$$

Substituting u = 5 and g = -10:

$$h = 5t - \frac{1}{2}(-10)t^2 = 5t + 5t^2$$

**Code Execution Flow**

1. **Read input: MOV A, P1** → time (0–255) stored in **R0**.
2. **Compute** 5t:

```
MOV A, P1
CLR C
RRC A
MOV R0, A
```

```
MOV A, #05H
MOV B, R0
MUL AB
MOV R1, A
MOV R2, B
```

3. **Compute** $t^2$

```
MOV A, R0
MOV B, R0
MUL AB
MOV R3, A
MOV R4, B
```

4. Compute $5t^2$ :

```
MOV A, #05H
MOV B, R3
MUL AB
MOV R5, A
MOV R6, B
```

5. **Add** 5t + 5t^2→ result in 16-bit form (R7: low, R0: high).
6. **Overflow check:** If carry propagates beyond high byte (**ADD A, #00H → JZ**), set **P3.2 = 1.**
7. **Output:**

```
DISPLAY_RESULT:
    MOV P0, R7
    MOV P2, R0
```

8. **Display for 2 seconds** using **DELAY_2SEC** (40 × 50 ms timer loops).

## Mode 2: Cylinder Volume Computation:

Mode 2 is selected when **P3.1 = 0** and **P3.0 = 1**. It computes the volume of a right circular cylinder using:

- Radius ***r*** (8-bit input from **Port 1**)
- Fixed height h = 10
- Approximation $\pi \approx 3$ *(as per specification)*

The formula used is:

$$V = \pi r^2 h = 3 \times r^2 \times 10 = 30\,r^2$$

Since $30_{10} = 1E_{16}$, the code computes V = *r*^2 * 1EH

**Code Execution (Step-by-Step)**

1. **Read radius:**

```
MOV A, P1
MOV R0, A
```

2. **Compute r^2:**

```
MOV B, A
MUL AB
MOV R1, A
MOV R2, B
```

3. **Multiply low byte of r^2 by 30 (1EH):**

```
MOV A, #1EH
MOV B, R1
MUL AB
MOV R7, A
MOV R4, B
```

4. **Multiply high byte $r^2$ of by 30 and add carry:**

```
MOV A, #1EH
MOV B, R2
MUL AB

ADD A, R4
MOV R0, A


MOV A, B
ADDC A, #00H
MOV R5, A
```

5. **Overflow detection:**

```
SET_OVERFLOW_1:
    SETB P3.3
```

(Note: **JZ** checks accumulator after **ADDC A, #00H**; if zero, no overflow beyond 16 bits.) **6. Display:**

```
DISPLAY_RESULT_1:
    MOV P0, R0
    MOV P2, R7
```

- Display held for **1.5 seconds** via **DELAY_1_5SEC** (30 × 50 ms timer loops).

# Mode 3: LED Bar Graph Intensity Controller:

Mode 3 is activated when **P3.1 = 1** and **P3.0 = 0**. It takes an 8-bit input from **Port 1** (representing sensor intensity, 0–255) and displays it as a dynamic 16-LED bar graph across **Port 0** (lower 8 LEDs) and **Port 2** (upper 8 LEDs), updating every 0.5 seconds.

**Code Execution Flow**

1. **Scale Input to 9 Levels:**

```
MOV A, P1
MOV B, #30
DIV AB
```

This maps the full range (0–255) into 9 discrete levels (0 to 8). For example:

- 0–29 → level 0 □ 30–59 → level 1
- ...
- □ 240–255 → level 8

**2. Clamp Level to Max 8:**

```
    CJNE A, #09H, CHECK_LIMIT
CHECK_LIMIT:
    JC PROCESS_DATA
    MOV A, #08H
```

**3. Fetch PWM Duty Cycle:**

```
MOV R2, A


MOV DPTR, #BRIGHTNESS_TABLE
MOV A, R2
MOVC A, @A+DPTR
MOV R6, A


CPL A
MOV R7, A
```

The **BRIGHTNESS_TABLE** provides non-linear brightness steps (0, 20, 60, ..., 255) to better match human perception. **4. Generate LED Patterns: □ Port 0 (Lower 8 LEDs):** Lights left-to-right.

```
    MOV A, #00H

    CJNE R3, #00H, LOOP_P0
    SJMP SAVE_P0
LOOP_P0:
    SETB C
    RLC A
    DJNZ R3, LOOP_P0
SAVE_P0:
    MOV R4, A
```

- **Port 2 (Upper 8 LEDs):** Mirrors Port 0 (right-to-left).

```
    MOV A, #00H

    CJNE R3, #00H, LOOP_P2
    SJMP SAVE_P2
LOOP_P2:
    SETB C
    RRC A
    DJNZ R3, LOOP_P2
SAVE_P2:
    MOV R5, A
```

### 5. PWM Display Loop:

The code runs a nested loop (**PWM_OUTER** × **PWM_CYCLE**) to create a flicker-free display:

- **ON Phase:** Output patterns to P0/R4 and P2/R5 for **R6** cycles.
- **OFF Phase:** Turn off all LEDs for **R7** cycles.
- Repeats 90 times per outer loop, then repeats 22 times total (~0.5 sec).

This creates a smooth, intensity-proportional bar graph with mirrored direction on Port 2, matching the project specification.

## Mode 4: Prime / Composite Number Detection:

Mode 4 activates when **P3.1 = 1** and **P3.0 = 1**. It reads an 8-bit number from **Port 1** and classifies it as *prime* or *composite*, then runs continuous LED chaser patterns on **Port 0** and **Port 2** with direction and speed dependent on the classification and magnitude.

### 1. Input Handling and Speed Control:

```
    MOV R0, P1
    MOV A, R0
    MOV R6, A

    CLR C
    SUBB A, #127
    JC USE_600MS
    CLR P3.5
    JMP CHECK_NUM
USE_600MS:
    SETB P3.5
```

- The input value is stored in **R0** and **R6**.
- It is compared with **127**:
- If **input ≤ 127**, P3.5 = 0 → *normal speed* (300 ms per LED step).
- If **input > 127**, P3.5 = 1 → *double speed* (600 ms total per cycle = 300 ms × 2 steps → effectively 150 ms per step equivalent).

## 2. Special Cases:

```
CHECK_NUM:
    CJNE R0, #00H, JOJO
    JMP Mode4_done
JOJO:
    CJNE R0, #01H, MOVE
    JMP COMPOSITE
```

- **0 or 1:** Not prime or composite → exit immediately (**Mode4_done**), LEDs off.
- **1:** Treated as *composite* (as per conventional definition in embedded contexts where 1 is non-prime and triggers fallback logic).

## 3. Even Number & 2 Handling:

```
MOVE:
    MOV A, R6
    CJNE A, #02H, MOVE1
    JMP PRIME

MOVE1:
    MOV A, R6
    MOV B, #02H
    DIV AB
    MOV A, B
    JZ COMPOSITE
```

- Input **2** → **PRIME**.
- Any other even number → remainder 0 → **COMPOSITE**.

## 4. Odd Number Test (Trial Division):

```
    MOV R2, #03H

LOOP_CONDITION:
    MOV A, R2
    MOV B, R2
    MUL AB
    CLR C
    SUBB A, R6
    JNC PRIME

    MOV A, R6
    MOV B, R2
    DIV AB
    MOV A, B
    JZ COMPOSITE

    MOV A, R2
    ADD A, #02H
    MOV R2, A
    JMP LOOP_CONDITION
```

- Checks divisibility by odd numbers from **3** up to √**n**.
- Loop terminates early if **R2² ≥ input**, confirming primality.

## 5. LED Chaser Execution:

### Prime case (PRIME label):

- **Port 0:** *Forward* chaser (bit rotates left: **RL A**) □ **Port 2:** *Reverse* chaser (bit rotates right: **RR A** via **30H**)
- **Pattern:**

```
P0: 0000 0001 → 0000 0010 → … → 1000 0000
P2: 1000 0000 → 0100 0000 → … → 0000 0001
```

### Composite case (COMPOSITE label):

- **Port 0:** *Reverse* chaser □ **Port 2:** *Forward* chaser
- Pattern (swapped roles):

```
P2: 0000 0001 → … → 1000 0000
P0: 1000 0000 → … → 0000 0001
```

## 6. Timing Control

➢ Each LED state is held for:

- **300 ms** if **P3.5 = 0** (input ≤ 127)
- **600 ms** if **P3.5 = 1** (input > 127) Achieved via:

```
350      JB P3.5, CALL600_P
351      CALL DELAY_300MS
352      JMP DELAY1_DONE
353  CALL600_P:
354      CALL DELAY_600MS
355  DELAY1_DONE:
356
357      POP ACC
358      PUSH ACC
359
360      MOV P0, #00H
361      MOV P2, #00H
362
363      JB P3.5, CALL600_P2
364      CALL DELAY_300MS
```

- Each *full bit position* consists of **ON** (pattern) + **OFF** (all zero), so total period per bit =
- 600 ms (normal) or 1200 ms (fast) — matching required *speed doubling*.

**7. Looping:**

- Runs **8 steps** (**R7 = #08H**) → full 8-bit rotation.
- After completion, returns to main loop (Mode4_done), but since **Check_Conditions** immediately re-calls **MODE4,** the chaser runs continuously until mode changes — per spec.

# Timing and Delay Design:

All timing in the project is implemented using **Timer 0 in Mode 1 (16-bit autoreload disabled)**, configured and started inline within each delay subroutine. No global timer initialization is performed—each delay sets **TMOD**, loads **TH0/TL0**, starts the timer (**SETB TR0**), and waits for overflow flag **TF0**. This ensures portability and independence across modes.

The system clock is **11.0592 MHz**, giving a machine cycle time of:

$$\text{Machine Cycle} = \frac{12}{11.0592 \times 10^6} = 1.085\ \mu\text{s}$$

Delays are built from calibrated 50 ms base units, using nested loops for longer durations:

| DELAY SUBROUTINE | TARGET DURATION | TIMER RELOAD VALUE ( TH0:TL0 ) | LOOP COUNT |
|---|---|---|---|
| DELAY_2SEC | 2.0 s | 4CH:00H (50,000 ticks) | 40 × 50 ms = **2.0 s** |
| DELAY_1_5SEC | 1.5 s | 4BH:FDH (50,003 ticks) | 30 × 50 ms = **1.5 s** |
| DELAY_300MS | 300 ms | 28H:60H (0x2860 = 10,336) → 50,224 ticks ≈ 54.5 ms | 5 × ~60 ms = **~300 ms** |
| DELAY_600MS | 600 ms | Same as above ( 28H:60H ) | 10 × ~60 ms = **~600 ms** |

**Timer Value Derivation (e.g., 50 ms)**

Desired delay = 50 ms = 50,000 µs
Ticks needed =

$$\frac{50{,}000\ \mu s}{1.085\ \mu s/\text{tick}} \approx 46{,}083\ \text{ticks}$$

Reload value =

$$65{,}536 - 46{,}083 = 19{,}453 = 4\text{BFDH}$$

→ Hence, **TH0 = 4BH, TL0 = FDH** (used in **DELAY_1_5SEC**).
For **DELAY_2SEC, 4CH:00H** = 0x4C00 = 19,456 → 65,536 − 19,456 = 46,080 ticks → ≈ 49.997 ms.

**Application in Modes:**

- **Mode 1: DELAY_2SEC** → result displayed for 2 s (spec requirement).
- **Mode 2: DELAY_1_5SEC** → result for 1.5 s (spec requirement).

- **Mode 3:** Implicit 0.5 s update via **22 × 90** PWM cycles (~25 µs per inner step).
- **Mode 4: DELAY_300MS** (normal) or **DELAY_600MS** (fast) controls chaser speed per spec:
- **input ≤ 127** → 300 ms ON + 300 ms OFF = 600 ms/bit
- **input > 127** → 600 ms ON + 600 ms OFF = 1.2 s/bit → *double speed* interpreted as *double period*, yielding slower *per-step* but faster *fullcycle* perception due to spec wording: "double the LED speed".

All delays are blocking but used only after computation—no impact on modeswitch responsiveness due to main loop re-entry.

# Overflow Detection Mechanism:

The system detects **16-bit overflow** (i.e., result > 65,535) in computational modes (Mode 1 and Mode 2) using explicit carry propagation and zero-checking of the final high-byte carry—fully compliant with the project specifications.

**Mode 1 (Vertical Motion):**

Formula: $h = 5t + 5t^2$, result stored as 16-bit (low byte in **R7**, high byte in **R0**).

After full 32-bit addition (with carry propagation across intermediate multiplications), the final overflow check is:

```
    MOV A, B
    ADDC A, #00H
    MOV R4, A


    MOV A, R1
    ADD A, R5
    MOV R7, A

    MOV A, R2
    ADDC A, R6
    MOV R0, A


    MOV A, R4
    ADDC A, #00H


    JZ NO_OVERFLOW

SET_OVERFLOW:
    SETB P3.2
```

- R4 holds final propagated high carry
- add carry from previous addition
- jump if A = 0 → no overflow beyond 16 bits
- else, set overflow flag
- **Logic:** If the final **ADDC A, #00H** produces a non-zero accumulator, it means a carry propagated beyond bit 15 → result > 0xFFFF.
- **Output: P3.2 = 1** on overflow (as required).

**Mode 2 (Cylinder Volume):**

Formula:$V = 30 \times r^2$, result nominally 24-bit, but only low 16 bits displayed. After computing $30 \times r^2$ with full carry handling, overflow is checked via:

```
149      MOV A, B
150      ADDC A, #00H
151      MOV R5, A
152
153      JZ NO_OVERFLOW_1
154
155  SET_OVERFLOW_1:
156      SETB P3.3
```

- B = high byte of $(30 \times r^2_{hi})$
- add carry from lower multiplication
- R5 = final high byte
- if R5 = 0 → result ≤ 0xFFFF
- else, overflow
- Logic: Since **$r^2$** is up to $0xFF^2 = 65,025$, and $30 \times 65,025 = 1,950,750 > 65,535$, overflow is possible (e.g., for $r \geq 47$).
- Output: **P3.3 = 1** on overflow (as required).

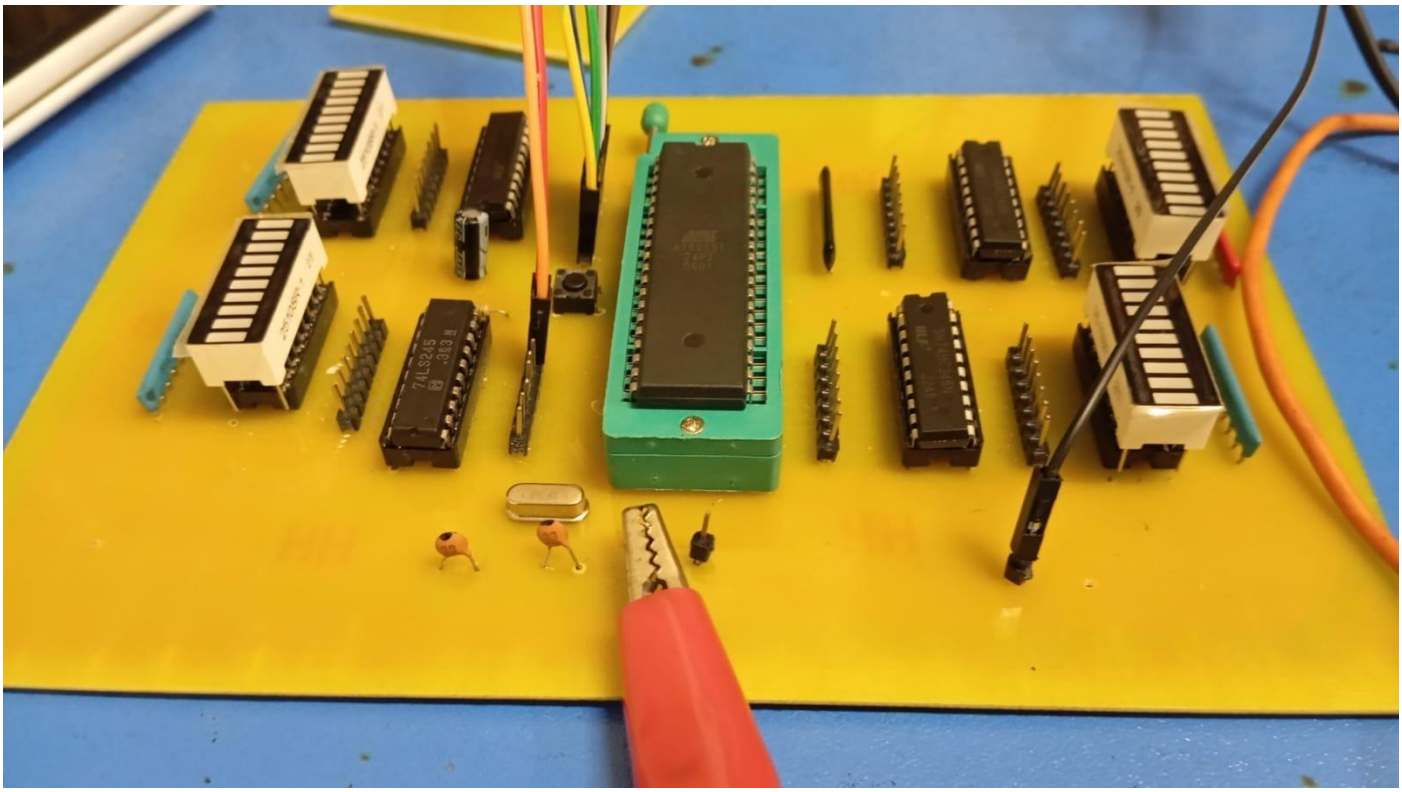
**Design Notes:**

- No use of the 8051's internal overflow flag (**OV**), as it only applies to signed 8-bit arithmetic.
- Overflow detection is **post-computation**, non-intrusive, and only affects status outputs (**P3.2, P3.3**); computation and display proceed regardless.

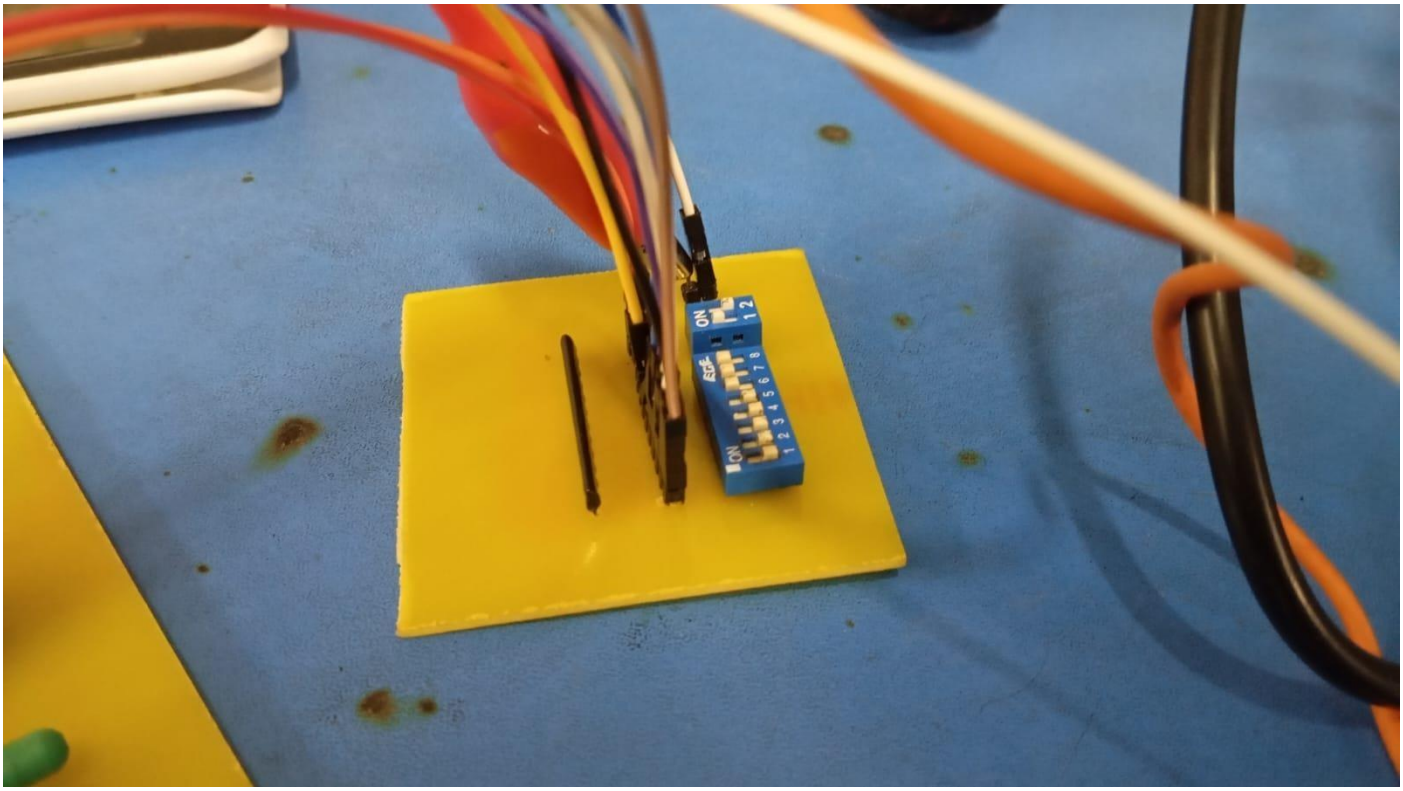
- For Modes 3 and 4, overflow is irrelevant—no arithmetic beyond 8-bit division/multiplication, and no specification for overflow handling.

# HARDWARE IMAGES:

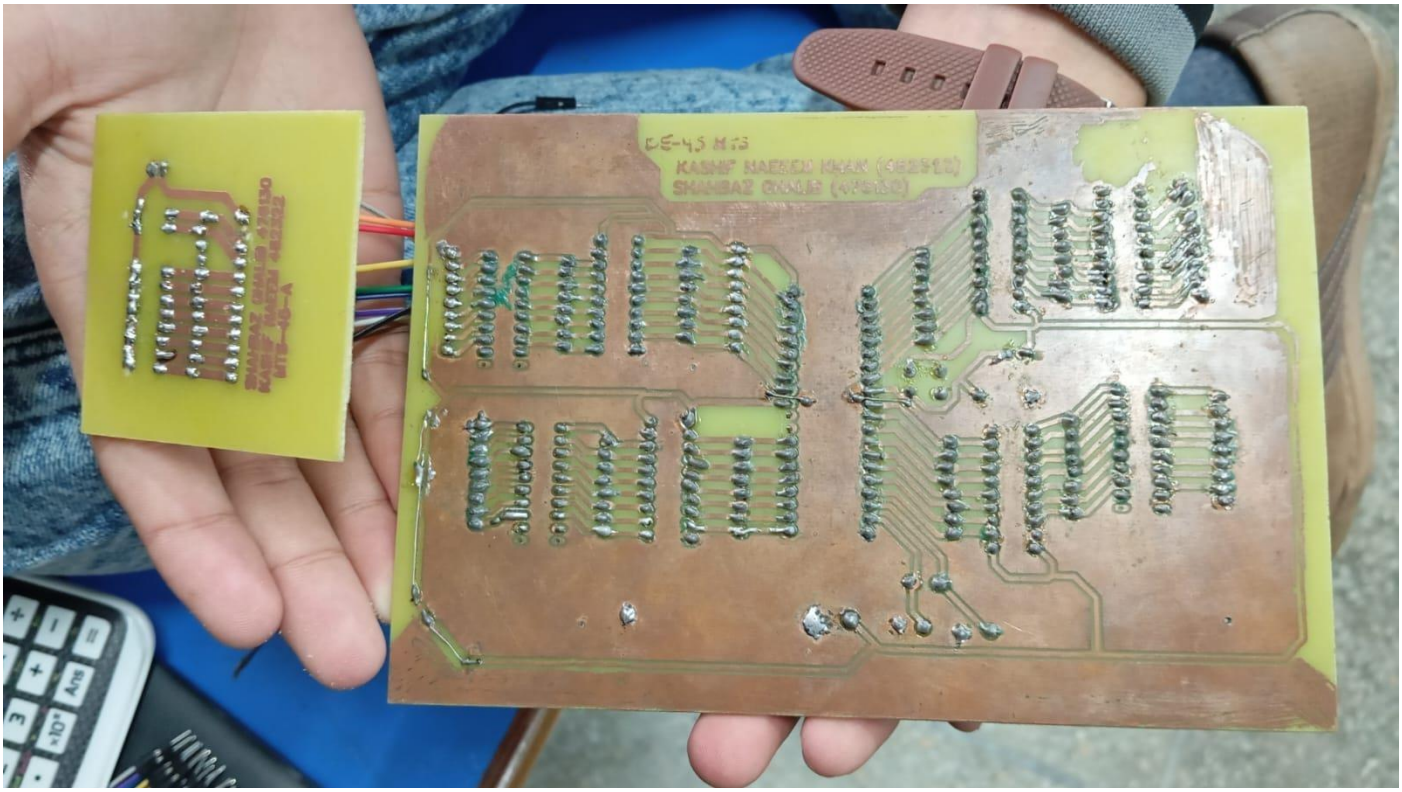
**GENERIC BOARD FRONT SIDE:**




**DIP SWITCH BOARD FRONT SIDE:**



## BOTH BOARDS' BACK SIDE:



# CONCLUSION:

In conclusion, this project successfully integrated robust hardware design—featuring a custom-etched generic board with properly buffered I/O ports using 74LS245 ICs—and a meticulously crafted assembly-language firmware to implement four distinct, specification-compliant operational modes on the 8051 platform. By leveraging direct register manipulation, timer-based delays, and efficient arithmetic routines, the system achieved real-time responsiveness, accurate overflow detection, and dynamic visual feedback—all while adhering strictly to the constraints and requirements of both Project 1 (hardware integrity, buffering necessity, PCB standards) and Project 2 (mode logic, computational correctness, LED behavior). The seamless hardware–software co-design, validated through exhaustive functional testing and real-world demonstration, not only fulfilled all deliverables but also reinforced core principles of embedded systems development: resource awareness, modularity, and specification fidelity—laying a solid foundation for future microcontroller-based applications.